

Innovation University – Faculty of Computer and Information Technology

Gym Management & Database Administration System

Team members:

Mennatallah Mohamed El Sayed (24030258)

Sama Mohsen Hussien (24030191)

Dalia Mohamed Zaki (24030172)

Hana Hani Mostafa (24030166)

Malak Mohamed (24030212)

Supervised by:

Dr. Marwa for the Database course

Table of Contents

1. Introduction
2. Database Schema & Architecture
3. Entity Relationship Diagram (ERD)
4. Database Implementation (SQL DDL & DML)
5. Advanced Database Objects (Procedures, Triggers, Functions, and Views)
6. Conclusion

1. Introduction

In modern fitness centers, managing member data, tracking attendance, scheduling classes, and monitoring financial subscriptions manually can lead to inefficiencies and data redundancy. This project presents a comprehensive **Gym Management System** built on a robust relational database backend using MySQL.

The main objective is to automate gym operations by structuring data related to members, trainers, membership plans, payments, and daily attendance. The database enforces standard relational constraints to guarantee data integrity, supports automated security controls through database triggers, and simplifies reporting using customized SQL views and functions.

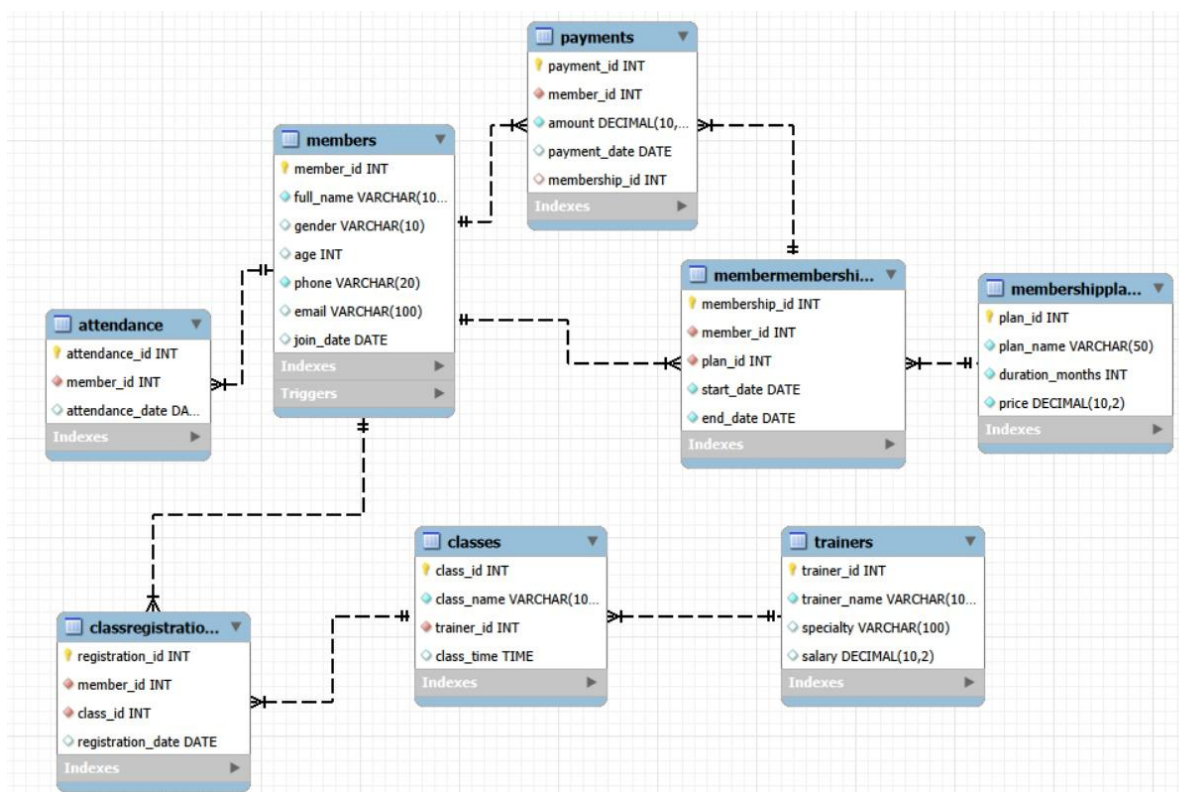
2. Database Schema & Architecture

To accommodate the gym workflow, the system consists of **8 interconnected tables**:

- **Members:** Stores personal information about gym clients, implementing data constraints for age limitations and ensuring contact uniqueness (Phone/Email).
- **Trainers:** Contains professional profiles of coaches, including their specialty and monthly salaries.
- **MembershipPlans:** Defines the available packages, subscription periods (in months), and rates.
- **MemberMemberships:** A bridge table handling the many-to-many relationship between Members and Plans, tracking activation and expiry dates.
- **Payments:** Records all billing and transaction logs linked directly to members and their specific membership packages.
- **Attendance:** Monitors daily check-ins for gym members using automated timestamps.
- **Classes:** Schedules group sessions (e.g., Yoga, Bodybuilding) and assigns them to specific trainers and time slots.
- **ClassRegistrations:** Manages member enrollments in individual fitness classes.

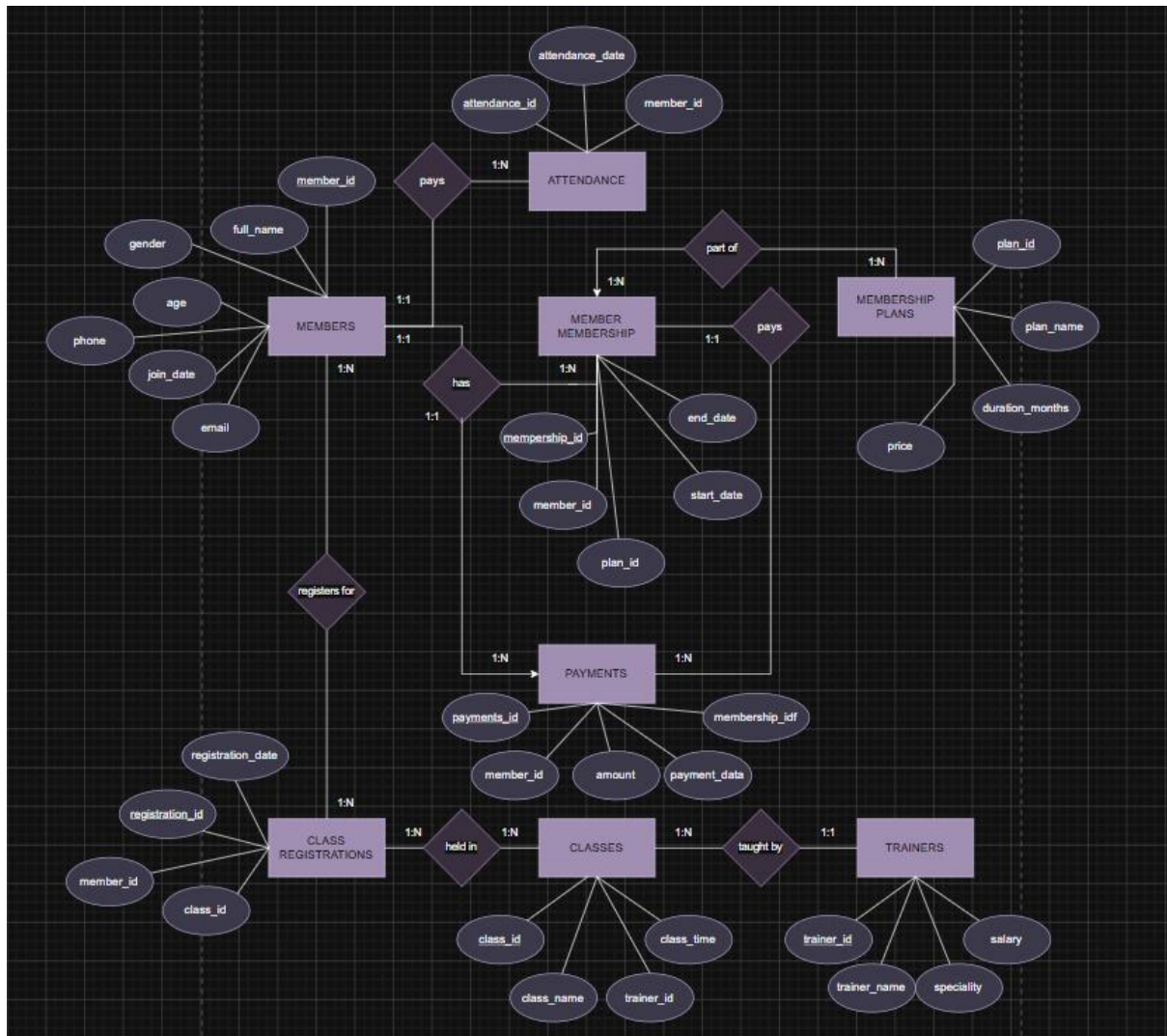
A. Relational Schema Mapping

- * Members (member_id, full_name, gender, age, phone, email, join_date)
- * Trainers (trainer_id, trainer_name, specialty, salary)
- * MembershipPlans (plan_id, plan_name, duration_months, price)
- * MemberMemberships (membership_id, member_id, plan_id, start_date, end_date)
- * Payments (payment_id, member_id, membership_id, amount, payment_date)
- * Attendance (attendance_id, member_id, attendance_date)
- * Classes (class_id, class_name, trainer_id, class_time)
- * ClassRegistrations (registration_id, member_id, class_id, registration_date)



3. Entity Relationship Diagram (ERD)

The Entity-Relationship Diagram (ERD) serves as the conceptual blueprint of the Gym Management System. It visually represents the system's logical entities, their respective attributes, primary identifiers, and the specific structural relationships that govern the gym's database logic.

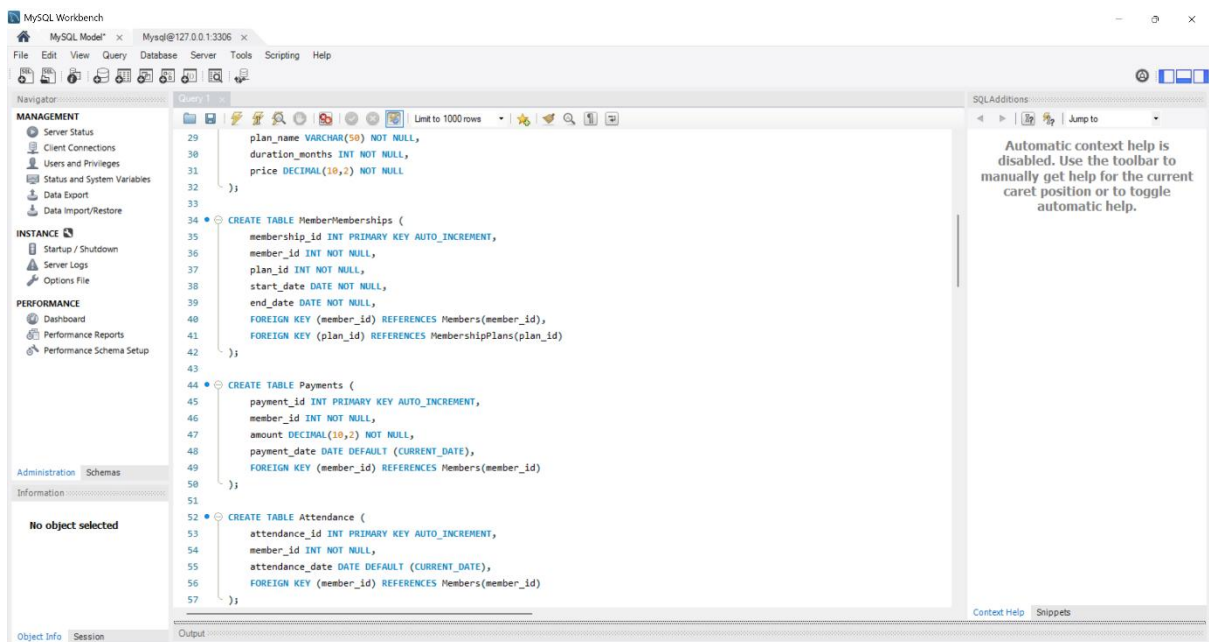
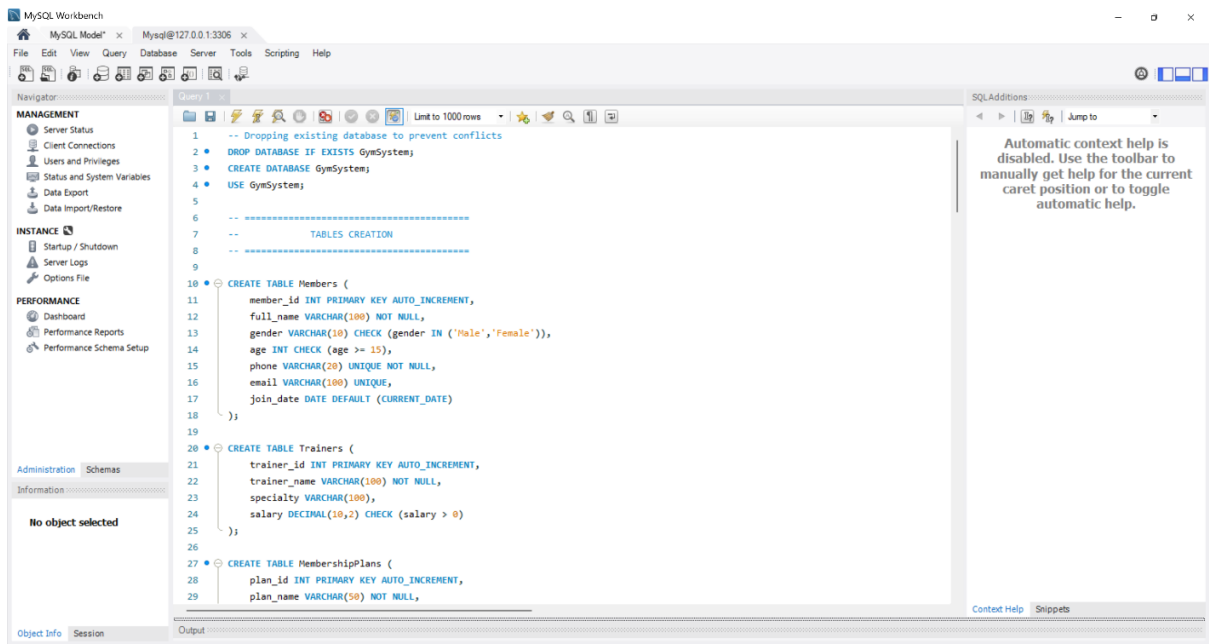


Database Relationship Description:

To enforce maximum data consistency and prevent structural redundancies, the diagram establishes the following cardinality rules:

- **Members to Attendance (\$1:N\$):** A single gym member can have multiple attendance check-in logs over time, while each attendance record belongs to exactly one specific member.
- **Members and Membership Plans to Member Membership (\$1:1\$ to \$1:N\$):** This acts as an associative entity to resolve the many-to-many subscription cycle. A member can subscribe to multiple sequential plans over time, and a plan can be assigned to multiple active members.
- **Member Memberships to Payments (\$1:N\$):** To allow flexible or split financial transactions, each subscription record can have one or more linked payment logs.
- **Trainers to Classes (\$1:1\$ to \$1:N\$):** A specialized trainer can coach and teach multiple scheduled gym classes, but each group class session is assigned to exactly one primary coach.
- **Members and Classes to Class Registrations (\$1:1\$ to \$1:N\$):** Manages group fitness enrollments. Each class session can hold multiple registered members, and each member can sign up for various classes simultaneously.

4. Database Implementation (SQL DDL & DML)



MySQL Workbench

MySQL Model x MySQL@127.0.0.1:3306 x

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

Object Info Session Output

```
59 CREATE TABLE Classes (  
60   class_id INT PRIMARY KEY AUTO_INCREMENT,  
61   class_name VARCHAR(100) NOT NULL,  
62   trainer_id INT NOT NULL,  
63   class_time TIME,  
64   FOREIGN KEY (trainer_id) REFERENCES Trainers(trainer_id)  
65 );  
66  
67 CREATE TABLE ClassRegistrations (  
68   registration_id INT PRIMARY KEY AUTO_INCREMENT,  
69   member_id INT NOT NULL,  
70   class_id INT NOT NULL,  
71   registration_date DATE DEFAULT (CURRENT_DATE),  
72   FOREIGN KEY (member_id) REFERENCES Members(member_id),  
73   FOREIGN KEY (class_id) REFERENCES Classes(class_id)  
74 );  
75  
76 -- Schema Modification: Linking Payments directly to specific Memberships  
77 ALTER TABLE Payments  
78 ADD COLUMN membership_id INT,  
79 ADD FOREIGN KEY (membership_id) REFERENCES MemberMemberships(membership_id);  
80  
81 -- =====  
82 -- DATA INSERTION (DML)  
83 -- =====  
84  
85 INSERT INTO Members (full_name, gender, age, phone, email) VALUES  
86 ('Ahmed Ali', 'Male', 22, '01011111111', 'ahmed@gmail.com'),  
87 ('Sara Mohamed', 'Female', 28, '01022222222', 'sara@gmail.com');
```

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

MySQL Workbench

MySQL Model x MySQL@127.0.0.1:3306 x

File Edit View Query Database Server Tools Scripting Help

Navigator

MANAGEMENT

- Server Status
- Client Connections
- Users and Privileges
- Status and System Variables
- Data Export
- Data Import/Restore

INSTANCE

- Startup / Shutdown
- Server Logs
- Options File

PERFORMANCE

- Dashboard
- Performance Reports
- Performance Schema Setup

Administration Schemas

Information

No object selected

Object Info Session Output

```
88 ('Wona Khaled', 'Female', 24, '01033333333', 'wona@gmail.com');  
89  
90 INSERT INTO Trainers (trainer_name, specialty, salary) VALUES  
91 ('Captain Omer', 'Bodybuilding', 12000),  
92 ('Captain Nada', 'Fitness', 10000);  
93  
94 INSERT INTO MembershipPlans (plan_name, duration_months, price) VALUES  
95 ('Monthly', 1, 500),  
96 ('3 Months', 3, 1200),  
97 ('Annual', 12, 4000);  
98  
99 INSERT INTO MemberMemberships (member_id, plan_id, start_date, end_date) VALUES  
100 (1, 1, '2026-05-01', '2026-06-01'),  
101 (2, 2, '2026-05-01', '2026-08-01');  
102  
103 INSERT INTO Payments (member_id, membership_id, amount) VALUES  
104 (1, 1, 500),  
105 (2, 2, 1200),  
106 (1, 1, 300);  
107  
108 INSERT INTO Attendance (member_id) VALUES (1), (2), (1);  
109  
110 INSERT INTO Classes (class_name, trainer_id, class_time) VALUES  
111 ('Yoga', 2, '18:00:00'),  
112 ('Muscle Building', 1, '20:00:00');  
113  
114 INSERT INTO ClassRegistrations (member_id, class_id) VALUES (1, 1), (2, 2), (3, 1);  
115  
116 -- Data Modification Examples
```

SQLAdditions

Automatic context help is disabled. Use the toolbar to manually get help for the current caret position or to toggle automatic help.

Context Help Snippets

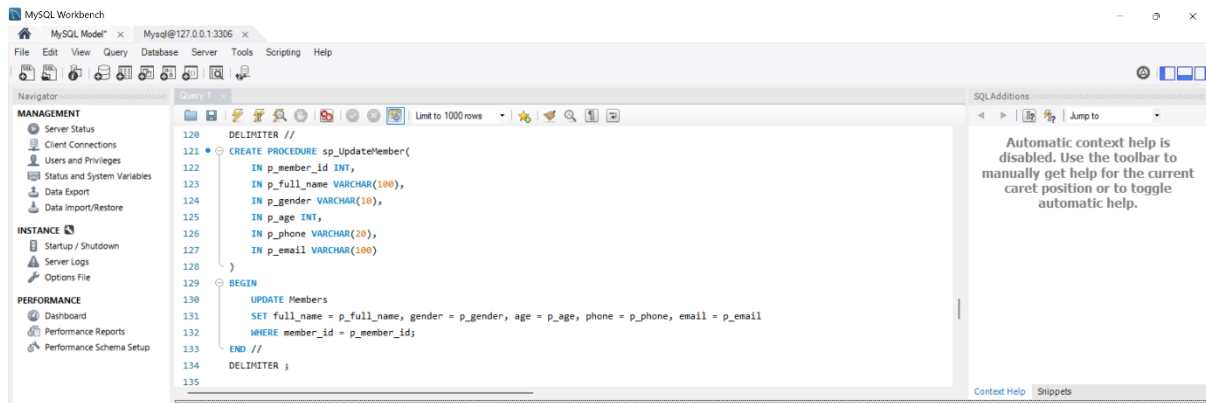
Object Info Session Output

```
116 -- Data Modification Examples  
117 UPDATE Members SET phone = '01099999999' WHERE member_id = 1;  
118 DELETE FROM Attendance WHERE attendance_id = 1;
```

5. Advanced Database Objects

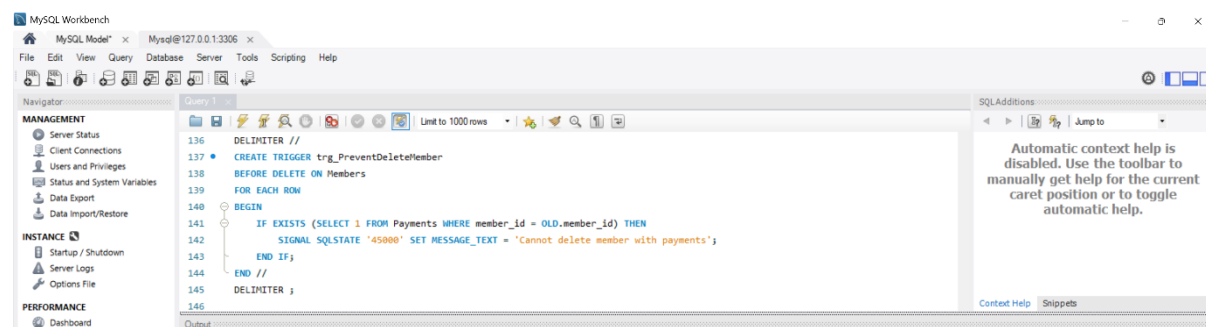
A. Stored Procedure (sp_UpdateMember)

To isolate client-side logic from direct table modification, a stored procedure was created to handle updating member profiles securely.



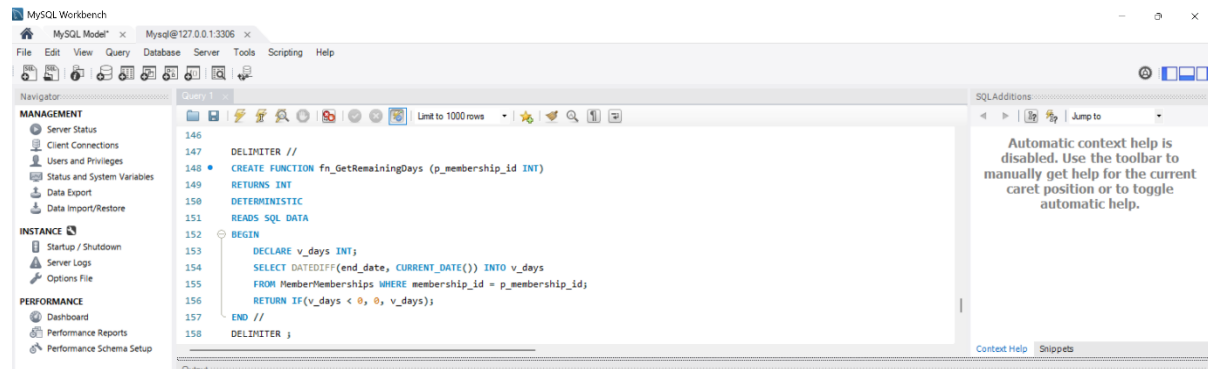
B. Database Trigger (trg_PreventDeleteMember)

To preserve financial logs and database history, this constraint trigger blocks the deletion of any member who has an active billing or payment profile.



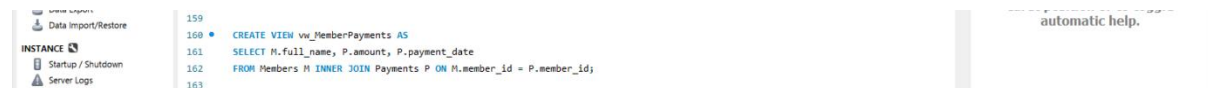
C. User-Defined Scalar Function (fn_GetRemainingDays)

An automated system function that calculates the remaining validity days of a member's current subscription.



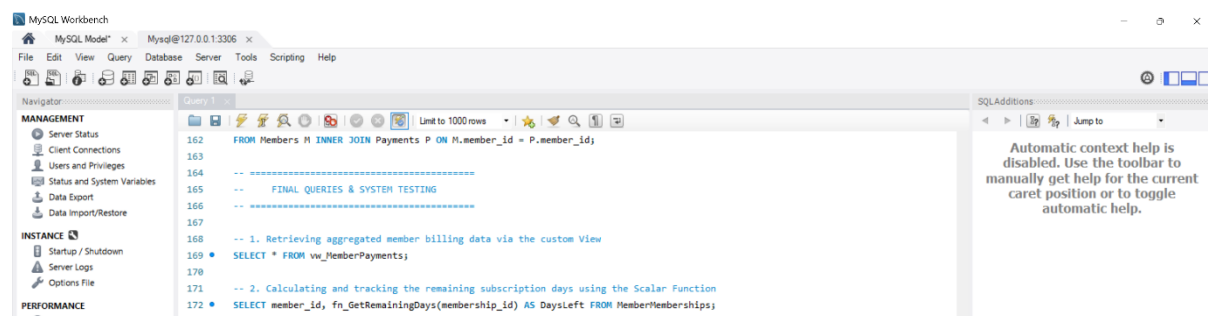
D. Database View (vw_MemberPayments)

Simplifies manager dashboards by abstracting entity joins and directly outputting customer billing metrics.



E. Final Queries Execution & System Testing

To validate the database architecture, ensure data consistency, and test the advanced database objects (the created Views and Functions), the following operational queries are executed:



6. Conclusion

The Gym Management Database System successfully delivers a scalable, structured, and consistent foundation for fitness facility workflows. By prioritizing relational integrity and leveraging professional SQL concepts—such as foreign keys, data constraints, automated functions, triggers, and aggregated views—the backend effectively prevents data redundancy while securing historical transactions. This engine is fully optimized to integrate with any target application or front-end platform dashboard.